Al-Azhar UNIVERSITY

Faculty of Engineering

Computers and Systems Engineering Department

EXPERIMENT 6 – Client/Server Application with Datagrams Using C#

OBJECTIVES

- Upon completion of this lab, you will be able to:
- Implement client/server application using C#.

MATERIALS/EQUIPMENT NEEDED

1. Visual Studio Software.

INTRODUCTION

Connection-Oriented vs. Connectionless Communication

There are two primary approaches to communicating between applications **connection oriented** and **connectionless**. Connection-oriented communications are similar to the telephone system, in which a connection is established and held for the length of the session. Connectionless services are similar to the postal service, in which two letters mailed from the same place and to the same destination may actually take two dramatically different paths through the system and even arrive at different times, or not at all.

In a connection-oriented approach, computers send each other control information through a technique called **handshaking** to initiate an end-to-end connection. The Internet is an **unreliable network**, which means that data sent across the Internet may be damaged or lost. Data is sent in **packets**, which contain pieces of the data along with information that helps the Internet route the packets to the proper destination. The Internet does not guarantee anything about the packets sent; they could arrive corrupted or out of order, as duplicates or not at all. The Internet makes only a "best effort" to deliver packets. A connection-oriented approach ensures reliable communications on unreliable networks, guaranteeing that sent packets will arrive at the intended receiver undamaged and be reassembled in the correct sequence.

In a connectionless approach, the two computers do not handshake before transmission, and reliability is not guaranteed data sent may never reach the intended recipient. A connectionless approach, however, avoids the overhead associated with handshaking and enforcing reliability less information often needs to be passed between the hosts.

Protocols for Transporting Data

There are many protocols for communicating between applications. Protocols are sets of rules that govern how two entities should interact. There are two protocols ,Transmission Control Protocol (TCP) and User Datagram Protocol (UDP). .NET's TCP and UDP networking capabilities are defined in the `System.Net.Sockets` namespace.

Transmission Control Protocol (TCP) is a connection-oriented communication protocol which guarantees that sent packets will arrive at the intended receiver undamaged and in the correct sequence. TCP allows protocols like HTTP to send information across a network as simply and reliably as writing to a file on a local computer. If packets of information don't arrive at the recipient, TCP ensures that the packets are sent again. If the packets arrive out of order, TCP reassembles them in the correct order transparently to the receiving application. If duplicate packets arrive, TCP discards them.

Applications that do not require TCP's reliable end-to-end transmission guaranty typically use the connectionless **User Datagram Protocol (UDP)**. UDP incurs the minimum overhead necessary to communicate between applications. UDP makes no guarantees that packets, called datagrams, will reach their destination or arrive in their original order.

There are benefits to using UDP over TCP. UDP has little overhead because UDP datagrams do not need to carry the information that TCP packets carry to ensure reliability. UDP also reduces network traffic relative to TCP due to the absence of handshaking, retransmissions, etc.

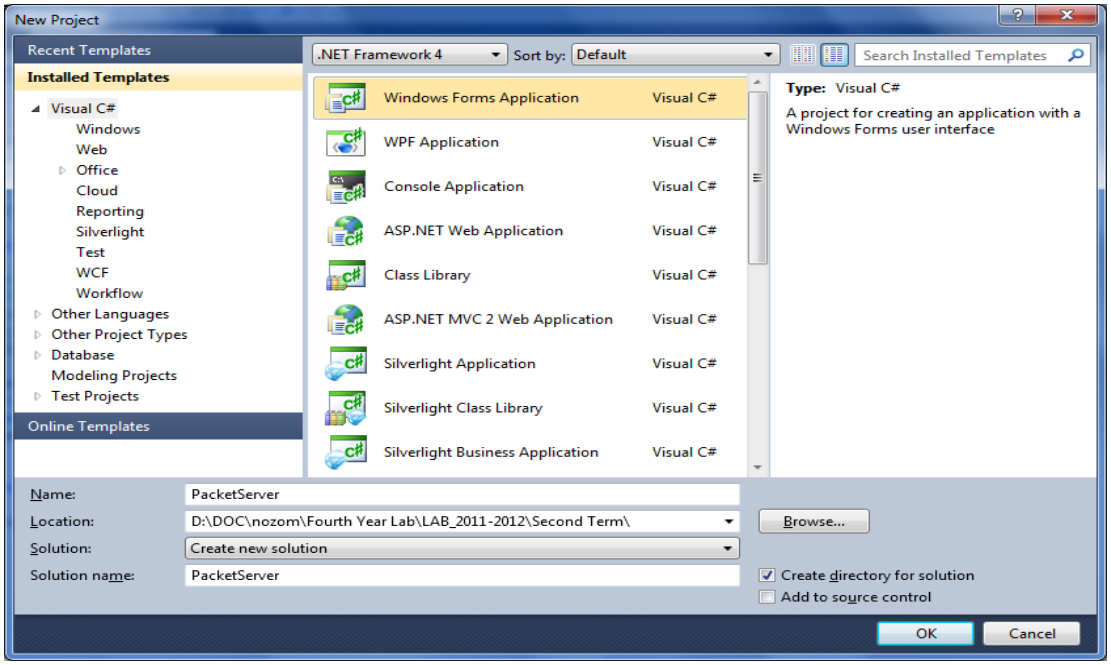
Connectionless Client/Server Interaction with Datagrams

Connectionless transmission via datagrams resembles the method by which the postal service carries and delivers mail. Connectionless transmission bundles and sends information in packets called datagrams, which can be thought of as similar to letters you send through the mail. If a large message will not fit in one envelope, that message is broken into separate message pieces and placed in separate, sequentially numbered envelopes. All the letters are mailed at once. The letters might arrive in order, out of order or not at all. The person at the receiving end reassembles the message pieces in sequential order before attempting to interpret the message. If the message is small enough to fit in one envelope, the sequencing problem is eliminated, but it is still possible that the message will never arrive. (Unlike with postal mail, duplicate of datagrams could reach receiving computers.) C# provides the UdpClient class for connectionless transmission. The UdpClient methods Send and Receive transmit data with Socket's SendTo method and read data with Socket's ReceiveFrom method, respectively

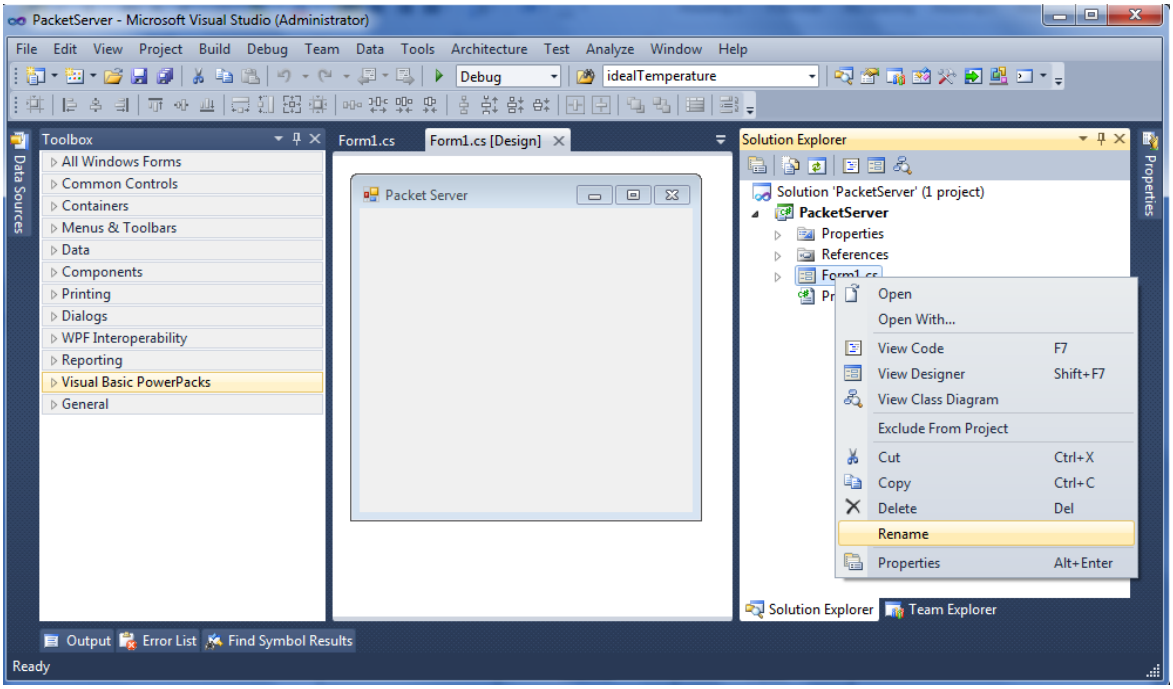
PROCEDURE

Task 1: Create Server side of Application

Step 1: From **start menu – All Programs -** run **Microsoft Visual Studio - Microsoft Visual Studio.** From **File** menu select **New Project**. At **New Project** window select **Windows Form Application** and its **Name** is **PacketServer** as shown in the following figure. Then press **OK**



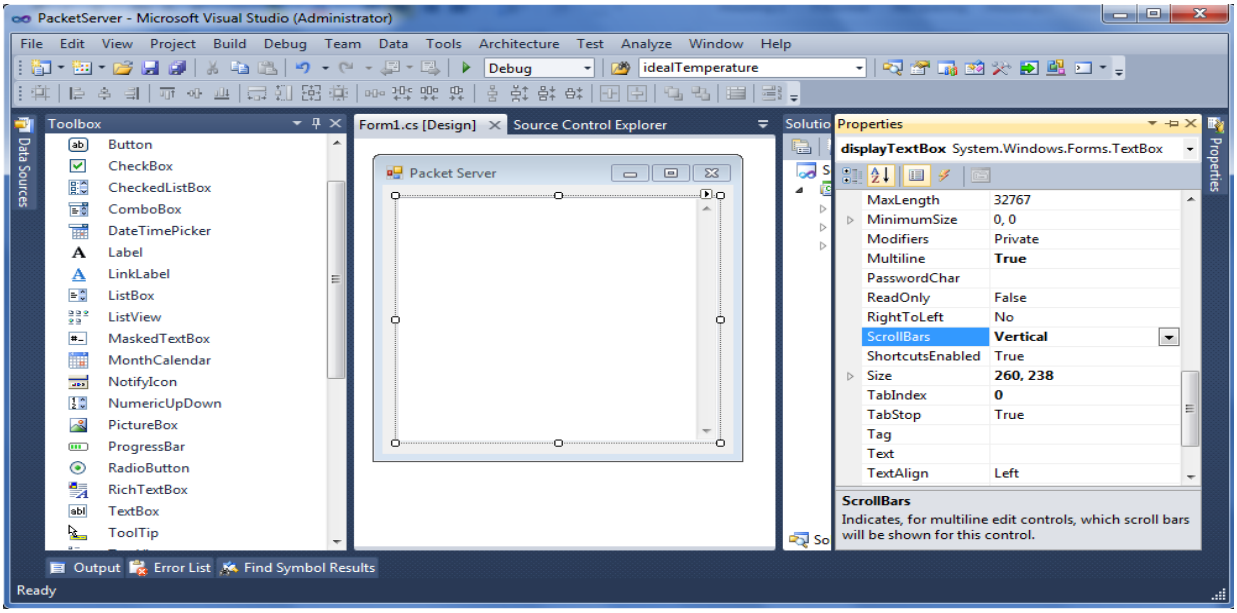
Step 2: On **Solution Explore window** right click on **Form1.cs** file then select **Rename** and change its name to be **PacketServerForm**. And change **Form Text** property to **Packet Server**.



Step 3: From the **Toolbox** window add the following controls

Control Type	Properties				
	Name	**Text**	**Multiline**	**ScrollBars**	**ReadOnly**
TextBox	displayTextBox		True	Vertical	True

After adding these controls, the result will be like this



Step 4: Right click on the **PacketServerForm** file then select **View Code**. Then change the code to be like this.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Net.Sockets;
using System.Net;
using System.Threading;

namespace PacketServer
{
    public partial class PacketServerForm : Form
    {
        private UdpClient client;
        private IPEndPoint receivePoint;

        public PacketServerForm()
        {
            InitializeComponent();
        }
     // method DisplayMessage sets displayTextBox's Text property
        // in a thread-safe manner
        private void DisplayMessage( string message )
        {
```

```
            // if modifying displayTextBox is not thread safe
            if ( displayTextBox.InvokeRequired )
            {


this.Invoke(new Action<string>(DisplayMessage), new object[] { message });

            } // end if
            else // OK to modify displayTextBox in current thread
               displayTextBox.Text += message;
         } // end method DisplayMessage

         // wait for a packet to arrive
         public void WaitForPackets()
         {
            while ( true )
            {
               // set up packet
               byte[] data = client.Receive( ref receivePoint );
              string strData ="\r\nClient: " +
      System.Text.Encoding.ASCII.GetString(data);
               DisplayMessage(strData);

               // echo information from packet back to client
               DisplayMessage( "\r\nEcho data back to client..." );
               client.Send( data, data.Length, receivePoint );
              DisplayMessage( "\r\nPacket sent" );
            } // end while
         } // end method WaitForPackets
      }
}
```

- The following lines define `DisplayMessage`, allowing any thread to modify `displayTextBox`'s `Text` property.

```
            private void DisplayMessage( string message )
            {
               if ( displayTextBox.InvokeRequired )
               {

      this.Invoke(new Action<string>(DisplayMessage), new object[] { message });
      );

               }
               else
                  displayTextBox.Text += message;
            }
```

- `PacketServerForm` method `WaitForPackets` executes an infinite loop while waiting for data to arrive at the `PacketServerForm`. When information arrives, `UdpClient` method `Receive` in line "**`byte[] data = client.Receive( ref receivePoint );`**" receives a `byte` array from the client. We pass to `Receive` the `IPEndPoint` object created in the constructor

this provides the method with an `IPEndPoint` to which the program copies the client's IP address and port number. This program will compile and run without an exception even if the reference to the `IPEndPoint` object is `null`, because method `Receive` initializes the `IPEndPoint` if it is `null`.

- The following lines update the `PacketServerForm`'s display to include the packet's information and content

```
string strData ="\r\nClient: " + System.Text.Encoding.ASCII.GetString(data);
DisplayMessage(strData);
```

- Line "**`client.Send( data, data.Length, receivePoint );`**" echoes the data back to the client, using `UdpClient` method `Send`. This version of `Send` takes three arguments the `byte` array to send, an `int` representing the array's length and the `IPEndPoint` to which to send the data. We use array `data` returned by method `Receive` as the data, the length of array `data` as the length and the `IPEndPoint` passed to method `Receive` as the data's destination. The IP address and port number of the client that sent the data are stored in `receivePoint`, so merely passing `receivePoint` to `Send` allows `PacketServerForm` to respond to the client.

Step 5: Double click on **PacketServerForm** form then write this code in **Load** event of form.

```
private void PacketServerForm_Load(object sender, EventArgs e)
        {
            client = new UdpClient( 50000 );
            receivePoint = new IPEndPoint( new IPAddress( 0 ), 0 );
            Thread readThread =
            new Thread( new ThreadStart( WaitForPackets ) );
            readThread.Start();
        }
```

- The "**`client = new UdpClient( 50000 );`**" line in the `Load` event creates an instance of the `UdpClient` class that receives data at port `50000`. This initializes the underlying `Socket` for communications.
- The "**`receivePoint = new IPEndPoint( new IPAddress( 0 ), 0 );`**" line creates an instance of class `IPEndPoint` to hold the IP address and port number of the client(s) that transmit to `PacketServerForm`. The first argument to the `IPEndPoint` constructor is an `IPAddress` object; the second argument is the port number of the endpoint. These values are both `0`, because we need only instantiate an empty `IPEndPoint` object. The IP addresses and port numbers of clients are copied into the `IPEndPoint` when datagrams are received from clients.

Step 6: In **FormClosing** event on **PacketServerForm** write this code.

```
private void PacketServerForm_FormClosing(object sender, FormClosingEventArgs e)
        {
            System.Environment.Exit(System.Environment.ExitCode);

        }
```

Task 2: Create Client side of Application

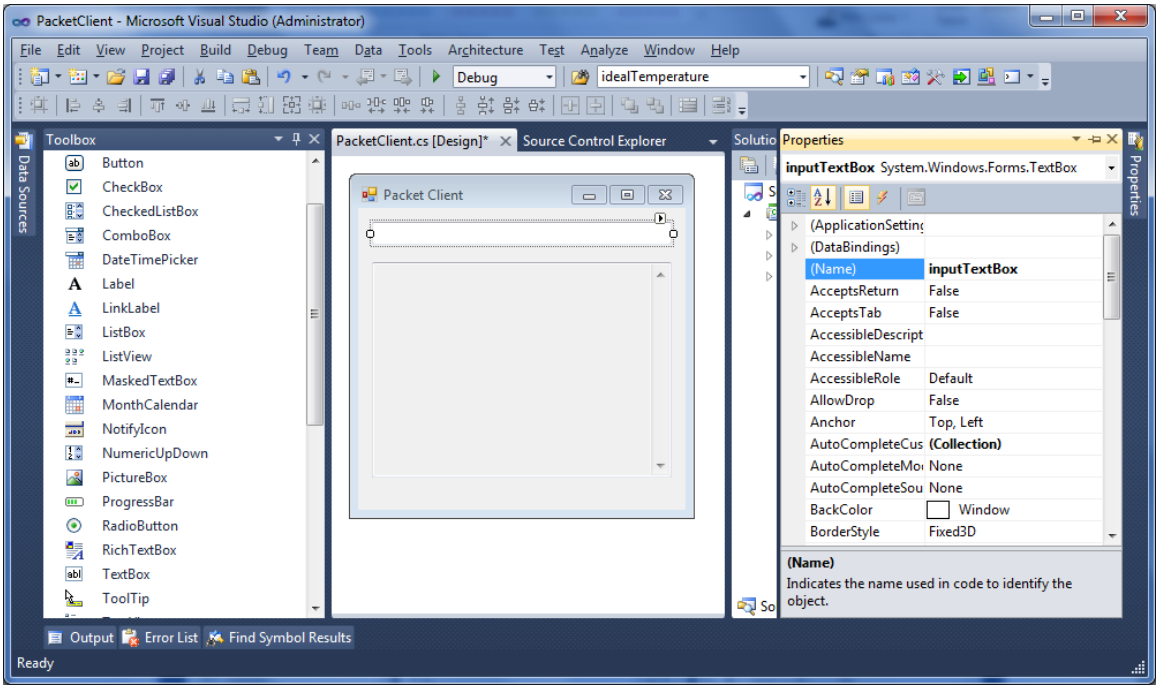
Step 1: From **start menu – All Programs -** run **Microsoft Visual Studio - Microsoft Visual Studio.** From **File** menu select **New Project**. At **New Project** window select **Windows Form Application** and its **Name** is **PacketClient**. Then press **OK**

Step 2: On **Solution Explore window** right click on **Form1.cs** file then select **Rename** and change its name to be **PacketClientForm**. And change **Form Text** property to **Packet Client**.

Step 3: From the **Toolbox** window add the following controls

Control Type	Properties				
	Name	Text	Multiline	ScrollBars	ReadOnly
TextBox	inputTextBox				
TextBox	displayTextBox		True	Vertical	True

After adding these controls the result will be like this



Step 4: Right click on the **PacketClientForm** then select **View Code**. Then change the code to be like this.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
```

```
using System.Windows.Forms;
using System.Net.Sockets;
using System.Net;
using System.Threading;


namespace PacketClient
{
    public partial class PacketClient : Form
    {
        private UdpClient client;
        private IPEndPoint receivePoint;


        public PacketClient()
        {
            InitializeComponent();
        }

        // method DisplayMessage sets displayTextBox's Text property
        // in a thread-safe manner
        private void DisplayMessage( string message )
        {
           // if modifying displayTextBox is not thread safe
           if ( displayTextBox.InvokeRequired )
           {
              // use inherited method Invoke to execute DisplayMessage

             this.Invoke(new Action<string>(DisplayMessage), new object[] { message });

           } // end if
           else // OK to modify displayTextBox in current thread
              displayTextBox.Text += message;
        } // end method DisplayMessage


        // wait for packets to arrive
        public void WaitForPackets()
        {
           while ( true )
           {
              // receive byte array from server
              byte[] data = client.Receive( ref receivePoint );

              // output packet data to TextBox
              string strData = System.Text.Encoding.ASCII.GetString(data);
              DisplayMessage("\r\n Server Echo: " + strData );
           } // end while
        } // end method WaitForPackets


    }
}
```

Method `WaiForPackets` of class `PacketClientForm` uses an infinite loop to wait for these packets. `UdpClient` method `Receive` blocks until a packet of data is received (line "**`byte[] data = client.Receive( ref receivePoint );`**"). The blocking performed by method `Receive` does not

prevent class `PacketClientForm` from performing other services (e.g., handling user input), because a separate thread runs method `WaitForPackets`.

Step 5: Double click on **PacketClientForm** form then write this code in Load event of form.

```
private void PacketClient_Load(object sender, EventArgs e)
        {
            receivePoint = new IPEndPoint( new IPAddress( 0 ), 0 );
            client = new UdpClient( 50001 );
            Thread thread =
            new Thread( new ThreadStart( WaitForPackets ) );
            thread.Start(); }
```

The line "**`client = new UdpClient( 50001 );`**" instantiates a `UdpClient` object to receive packets at port `50001`we choose port `50001` because the `PacketServer` already occupies port `50000`.

Step 6: In **KeyDown** event on **inputTextBox** write this code

```
private void inputTextBox_KeyDown(object sender, KeyEventArgs e)
        {
            if ( e.KeyCode == Keys.Enter )
           {
              // create packet (datagram) as string
              string packet = inputTextBox.Text;
              displayTextBox.Text +=
                 "\r\nSending packet containing: " + packet;

              // convert packet to byte array
              byte[] data = System.Text.Encoding.ASCII.GetBytes( packet );

              // send packet to server on port 50000
              client.Send( data, data.Length, "127.0.0.1", 50000 );
              displayTextBox.Text += "\r\nPacket sent\r\n";
              inputTextBox.Clear();
           } // end if

        }
```

- The `Client` object sends packets only when the user types a message in a `TextBox` and presses the Enter key. When this occurs, the program calls event handler `inputTextBox_KeyDown` which show above.
- The line "**`byte[] data = System.Text.Encoding.ASCII.GetBytes( packet );`**" converts the `string` that the user entered in the `TextBox` to a `byte` array.
- The line "**`client.Send( data, data.Length, "127.0.0.1", 50000 );`**" calls `UdpClient` method `Send` to send the `byte` array to the `PacketServerForm` that is located on `localhost` (i.e., the same machine). We specify the port as `50000`, which we know to be `PacketServerForm`'s port.

Step 6: In **FormClosing** event on **PacketClientForm** write this code.

```
private void PacketClient_FormClosing(object sender, FormClosingEventArgs e)
        {
            System.Environment.Exit(System.Environment.ExitCode);

        }
```